BASEFLUXMETHOD

January 9, 2026

[1]:
```
import redback
print(redback.__version__)
```

```
No module named 'lalsimulation'
lalsimulation is not installed. Some EOS based models will not work. Please use
bilby eos or pass your own EOS generation class to the model
14:57 bilby INFO    : Running bilby version: 2.3.0
14:57 redback INFO    : Running redback version: 1.12.1
```

```
1.12.1
```

[2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import redback.interaction_processes as ip
import redback.sed as sed
import redback.photosphere as photosphere
from astropy.cosmology import Planck18 as cosmo
import astropy.units as uu

import extinction
from extinction import ccm89, fitzpatrick99, apply, remove
from scipy.interpolate import RegularGridInterpolator
import sncosmo
```

[9]:
```
'''GRB EVENT ANALYSIS:'''
#STEP 1: Load in GRB event information eg. AB magnitude, redshift, filter,␣
 ↪frequency of filter?, Av if applicable
#GRB 060318
event_name = '060218'
AB_mag = 17.22
redshift = 0.0331
epoch = 11.0  # Days in observer frame
band = 'besselIr'
a_v = 0.39 #true value from literature

#STEP 2: Convert GRB event AB magnitude into flux density using redback
def calc_flux_density_from_ABmag(AB_mag):
    """
```

```
    Calculate flux density from AB magnitude assuming monochromatic AB filter

    :param magnitudes: AB magnitude values
    :return: flux density
    """
    return (AB_mag * uu.ABmag).to(uu.mJy)

flux_density_event = calc_flux_density_from_ABmag(AB_mag)

print("This is the flux density of  GRB event", event_name ,f"without␣
 ↪extinction correction: {flux_density_event:.3f}")

#check this using online calculator -- CORRECT -- OK TO PROCEED TO NEXT STEP

#STEP 3: Use fitzpatrick99 extinction function (if applicable) to find␣
 ↪de-reddened value for GRB flux density
#to get wavelength for GRB event, use redback tables
#wavelength must be an array
wavelength =  np.array([6498.09000]) #angstroms -- r-band wavelength
dereddened_flux_event = extinction.remove(fitzpatrick99(wavelength, a_v, 3.
 ↪1),flux_density_event)

#dereddened flux is now a 1-d array so display the first element
print(f"This is the extinction corrected GRB event flux density found using␣
 ↪fitzpatrick99: {dereddened_flux_event[0]:.3f}")

#to test if this is doing what it should be doing, if i increase the a_v value,␣
 ↪the flux should go upwards!
# if Av is changed to be higher 4-5 than the original value, flux density␣
 ↪should be getting brighter !!
#DOES THIS HAPPEN ? -- YES -- the value goes up !!
#PROCEED TO NEXT STEP

'''SN1998BW MODEL ANALYSIS:'''
#STEP 4: Define band from sncosmo to match the filter for the GRB event with␣
 ↪the frequency bandpass filter for input into SN1998bw model

print("GRB",event_name, "is associated with the filter band", band)

#we need to get frequency bandpass for the specific filter using sncosmo
bandpass = sncosmo.get_bandpass(band)
# wavelengths (in angstroms)
print(bandpass.wave)
#convert wavelength to m
wavelength_m = bandpass.wave *1e-10
#define c
```

```
c = 3.0e8
#now to get in terms of frequencies have to calculate using f = c / wavelength
frequencies_Hz = c / wavelength_m
#check + also do by hand -- DOES THIS WORK ? -- YES
print(frequencies_Hz)

# Calculate the central/effective frequency of the band
central_wavelength = bandpass.wave_eff # in Angstroms
central_frequency = c / (central_wavelength * 1e-10)

#STEP 5:

#calculate 1-D array of flux densities of 1998bw over time
#bug fix : ensure time is the same or related to the observed time
# i was working in two different frames which caused a 0 error

#define lambda_to_nu outside of the function: does this make the previous␣
 ↪frequencies_Hz obsolete ?

def lambda_to_nu(wavelength_angstrom):
    """ Converts wavelength in Angstroms to frequency in Hz """
    c = 299792458  # speed of light in m/s
    return c / (wavelength_angstrom * 1e-10)


#try time !!

time = np.geomspace(0.01,90,200)

#change the inner workings of the sn1998bw def
def sn1998bw_template(time, redshift, amplitude, **kwargs):
    """
    A wrapper to the SN1998bw template. Only valid between 1100-11000 Angstrom␣
 ↪and 0.01 to 90 days post explosion in rest frame

    Parameters
    ----------
    time
        time in days in observer frame (post explosion)
    redshift
        redshift
    amplitude
        amplitude scaling factor, where 1.0 is the original brightness of␣
 ↪SN1998bw; and f_lambda is scaled by this factor
    kwargs
        Additional keyword arguments required by redback.
    frequency
```

```
        Required if output_format is 'flux_density'. frequency to calculate -␣
↪Must be same length as time array or a single number).
    bands
        Required if output_format is 'magnitude' or 'flux'.
    output_format
        'flux_density', 'magnitude', 'spectra', 'flux', 'sncosmo_source'
    cosmology
        Cosmology to use for luminosity distance calculation. Defaults to␣
↪Planck18. Must be a astropy.cosmology object.
    Returns
    -------
        set by output format - 'flux_density', 'magnitude', 'spectra', 'flux',␣
↪'sncosmo_source'

    """
    import sncosmo
    model = sncosmo.Model(source='v19-1998bw')
    original_redshift = 0.0085 #redshift of 1998bw
    cosmology = kwargs.get("cosmology", cosmo)
    original_dl = (43*uu.Mpc).to(uu.cm).value

    # From roughly matching to Galama+ or Clocchiatti+1998bw light curves
    original_peak_time = 15
    model.set(z=original_redshift, t0=original_peak_time)
    model.set_source_peakmag(14.25, band='bessellb', magsys='ab')
    lls = np.linspace(1620, 11000, 300)
    f_lambda = model.flux(time, lls) #erg/s/cm^2/Angstrom.
    l_lambda = f_lambda * 4 * np.pi * original_dl**2  # erg/s/Angstrom

    # We consider this the rest frame spectrum of 1998bw. Now we can redshift␣
↪it and scale it.
    time_obs = time * (1 + redshift)
    lambda_obs = lls * (1 + redshift)
    dl_new = cosmology.luminosity_distance(redshift).cgs.value
    f_lambda_obs = l_lambda / (4 * np.pi * dl_new**2)
    f_lambda_obs = amplitude * f_lambda_obs * (1 + redshift) # accounting for␣
↪bandwidth stretching
    f_lambda_obs = f_lambda_obs * uu.erg / uu.s / uu.cm ** 2 / uu.Angstrom
    if kwargs['output_format'] == 'flux_density':
        frequency = kwargs['frequency']
        # work in obs frame
        ff_array = lambda_to_nu(lambda_obs)

        # Convert flux density to mJy
        fmjy = f_lambda_obs.to(uu.mJy, equivalencies=uu.
↪spectral_density(wav=lambda_obs * uu.Angstrom)).value
```

```
        # Create interpolator on obs frame grid
        flux_interpolator = RegularGridInterpolator(
            (time_obs, ff_array),
            fmjy,
            bounds_error=False,
            fill_value=0.0)

        #THIS IS WHERE I GET LOST -------------------------
        #ENSURE I AM ALWAYS WORKING IN THE OBSERVER FRAME ?
        # Prepare points for interpolation
        #change time to time_obs too ! This didnt change my code ?
        if isinstance(frequency, (int, float)):
            frequency = np.ones_like(time_obs) * frequency

        # Create points for evaluation -- CORRECT OR NO ?
        #MODIFIED !!
        #points = np.column_stack((time, frequency))
        # FIX: Align the query points with the grid (time_obs)
        points = np.column_stack((time_obs, frequency))
        #-----------------------------------------------------
        # Return interpolated flux density with (1+z) correction for observer␣
 ↪frame
        return flux_interpolator(points)
    elif kwargs['output_format'] == 'spectra':
        return namedtuple('output', ['time', 'lambdas',␣
 ↪'spectra'])(time=time_obs,
                                                                          ␣
 ↪lambdas=lambda_obs,
                                                                          ␣
 ↪spectra=f_lambda_obs)
    else:
        return sed.get_correct_output_format_from_spectra(time=time,␣
 ↪time_eval=time_obs,
                                                          spectra=f_lambda_obs,␣
 ↪lambda_array=lambda_obs,
                                                          **kwargs)



# Just pass an array of two values to satisfy the interpolator's grid␣
 ↪requirement
target_epoch = epoch
times = np.array([target_epoch, target_epoch + 0.1])

#only takes in one specific frequency -- so what was the point in the sncosmo␣
 ↪bandpass ?!!!!!! WHAT WAS IT ? -- DO WE STILL NEED IT --
```

```
result = sn1998bw_template(
    time=times,
    redshift=redshift,
    amplitude=1.0,
    output_format='flux_density',
    frequency=central_frequency, # TRY CENTRAL FREQ NOT -- singular frequency␣
 ↪from redback tables I-band effective width in Hz OR multiple values ? WHICH␣
 ↪ONE WORKS ? --
    cosmology=cosmo
)

f_1998bw_interpolated = result[0] #flux for 1998bw in mJy (float?)

# result[0] is now exactly the flux at day 10.5
print(f"Flux density at day {target_epoch}: {f_1998bw_interpolated} mJy")

#SKIPPED STEP 6 -- INTERPOLATED IN ONE GO

'''FINAL RATIO:'''
#STEP 7: take value from STEP 3 and STEP 6 in ratio with one another to get␣
 ↪final result of flux ratio

f_1998bw_ratio = dereddened_flux_event / f_1998bw_interpolated

print(f"The final flux density ratio result of F_GRB / F_1998bw =␣
 ↪{f_1998bw_ratio[0]:.3f}")


#ensure final ratio is unitless
# Ensure both are in the same units
# 1. Convert GRB flux to mJy (since the SN function returns mJy)
# dereddened_flux_event comes from STEP 3
f_grb_mJy = dereddened_flux_event.to(uu.mJy).value[0]

# 2. Calculate the ratio using pure floats (both are now mJy)
f_ratio = f_grb_mJy / f_1998bw_interpolated

print(f"GRB Flux (mJy): {f_grb_mJy:.4f}")
print(f"SN Flux (mJy): {f_1998bw_interpolated:.4f}")
print(f"Final Ratio: {f_ratio:.3f}")

#when i change the points from time to time_obs in this it gives me a correct␣
 ↪ish value ??
# points = np.column_stack((time_obs, frequency))

#time check -- INCLUDE ?
```

```
'''
# 1. Ensure frequency matches the length of the time array
if isinstance(frequency, (int, float)):
    # Use time_obs here to keep lengths consistent
    frequency_array = np.ones_like(time_obs) * frequency
else:
    frequency_array = frequency

# 2. Use time_obs to query the grid
points = np.column_stack((time_obs, frequency_array))

# 3. Return the result
return flux_interpolator(points)'''
```

```
This is the flux density of  GRB event 060218 without extinction correction:
0.470 mJy
This is the extinction corrected GRB event flux density found using
fitzpatrick99: 0.620 mJy
GRB 060218 is associated with the filter band bessellr
[5500. 5600. 5700. 5800. 5900. 6000. 6100. 6200. 6300. 6400. 6500. 6600.
 6700. 6800. 6900. 7000. 7100. 7200. 7300. 7400. 7500. 8000. 8500. 9000.]
[5.45454545e+14 5.35714286e+14 5.26315789e+14 5.17241379e+14
 5.08474576e+14 5.00000000e+14 4.91803279e+14 4.83870968e+14
 4.76190476e+14 4.68750000e+14 4.61538462e+14 4.54545455e+14
 4.47761194e+14 4.41176471e+14 4.34782609e+14 4.28571429e+14
 4.22535211e+14 4.16666667e+14 4.10958904e+14 4.05405405e+14
 4.00000000e+14 3.75000000e+14 3.52941176e+14 3.33333333e+14]
Flux density at day 11.0: 0.7142024413347644 mJy
The final flux density ratio result of F_GRB / F_1998bw = 0.868 mJy
GRB Flux (mJy): 0.6201
SN Flux (mJy): 0.7142
Final Ratio: 0.868
```

[9]:
```
'\n# 1. Ensure frequency matches the length of the time array\nif
isinstance(frequency, (int, float)):\n    # Use time_obs here to keep lengths
consistent\n    frequency_array = np.ones_like(time_obs) * frequency \nelse:\n
frequency_array = frequency\n\n# 2. Use time_obs to query the grid\npoints =
np.column_stack((time_obs, frequency_array))\n\n# 3. Return the result\nreturn
flux_interpolator(points)'
```